

Relatório técnico

Como Executar o Projeto

Para iniciar todos os serviços e abrir o frontend em modo terminal (TUI), execute:

Caso queira acompanhar os logs em tempo real, utilize:

No primeiro terminal:

```
docker compose up
```

No segundo terminal:

```
docker compose run --rm chat_frontend
```

Relatório

1. Descrição completa da arquitetura desenvolvida

- A arquitetura desenvolvida é feita em 3 partes:
 - **AS (Authentication Server):** servidor Kerberos responsável por autenticar o usuário.
 - **TGS (Ticket Granting Server):** servidor Kerberos que dá acesso ao serviço que o usuário quer acessar.
 - **Serviço de Chat**, dividido em:
 - **Backend:** servidor REST responsável por comunicar com o banco SQLite e garantir as regras de negócio.
 - **Frontend:** TUI (Terminal User Interface) responsável por renderizar a interface.

2. Descrição detalhada da implementação do Servidor de Autenticação (Authentication Server – AS)

- O Authentication Server recebe o username do usuário a ser autenticado.
- Depois busca no banco de dados o usuário com aquele username único.
- Com o usuário encontrado, temos a sua chave, que foi gerada a partir do KDF (Função de Derivação de Chave).
- Depois geramos o TGT (Ticket Granting Ticket), criptografado com a chave compartilhada com o TGS, para que somente o AS e o TGS tenham acesso às informações do ticket. Ele contém: username do usuário, chave de sessão do usuário com o TGS e uma validade.
- Também geramos a chave de sessão do cliente com o TGS, mas criptografada com a chave do usuário, para que só quem tem a senha consiga descriptografá-la.
- Por fim, devolvemos ao cliente o TGT, a `K_client_tgs` cifrada com a chave dele e o salt (usado para re-derivar a chave a partir da senha digitada).

```
def handle_client(conn):
    raw = conn.recv(4096)
    request = json.loads(raw.decode())

    username = request.get("username")

    if not username:
        conn.sendall(json.dumps({"error": "username is required"}).encode())
        return

    try:
        key_user = db.get_user_key(username)
        salt = db.get_user_salt(username)
    except KeyError:
        conn.sendall(json.dumps({"error": "unknown user"}).encode())
```

```

        return

    k_client_tgs = gen_session_key()

    tgt = encrypt(
        config.AS_TGS_KEY,
        {
            "username": username,
            "k_client_tgs": k_client_tgs.hex(),
            "valid_until": time.time() + config.TICKET_LIFETIME,
        },
    )

    response = {
        "k_client_tgs_enc": encrypt(key_user, {"k_client_tgs": k_client_tgs.hex()}),
        "tgt": tgt,
        "salt": salt.hex(),
    }

    conn.sendall(json.dumps(response).encode())
    print(f"[AS] Resposta enviada para o cliente {username}")

```

3. Descrição detalhada da implementação do Servidor de Emissão de Tickets (Ticket Granting Server – TGS)

- O TGS recebe o TGT, o serviço que o usuário quer acessar e o autenticador (username e timestamp criptografados com a `K_client_tgs`, a chave de sessão obtida do AS).
- Depois verifica se o autenticador e o TGT são válidos.
- Depois pega a chave de criptografia do serviço solicitado.
- E gera o ticket de serviço, criptografado com essa chave, contendo:

- username do usuário;
- chave de sessão entre o cliente e o serviço;
- até quando o ticket é válido.
- Também é gerada a chave de sessão do serviço, criptografada com a `K_client_tgs`, para o cliente conseguir recuperá-la.
- Por fim, devolvemos o ticket de serviço (que só o chat consegue abrir com a `K_chat`) e a `K_client_service` cifrada com a `K_client_tgs`.

```
def handle_client(conn):
    raw = conn.recv(4096)
    request = json.loads(raw.decode())

    tgt = request.get("tgt")
    service = request.get("service")
    authenticator = request.get("authenticator")

    if not tgt or not service or not authenticator:
        conn.sendall(json.dumps(
            {"error": "tgt, service, and authenticator are
required"}).encode())
        return

    try:
        tgt_data = decrypt(config.AS_TGS_KEY, tgt)
    except Exception:
        conn.sendall(json.dumps({"error": "invalid tgt"}).
encode())
        return

    k_client_tgs = bytes.fromhex(tgt_data["k_client_tgs"])

    try:
        auth_data = check_authenticator(k_client_tgs, auth
enticator)
```

```

except Exception:
    conn.sendall(json.dumps({"error": "invalid authenticator"}).encode())
    return

if auth_data.get("username") != tgt_data.get("username"):
    conn.sendall(json.dumps({"error": "invalid authenticator"}).encode())
    return

if not check_expiration(tgt_data.get("valid_until")):
    conn.sendall(json.dumps({"error": "expired tgt"}).encode())
    return

try:
    key_service = db.get_service_key(service)
except KeyError:
    conn.sendall(json.dumps({"error": "unknown service"}).encode())
    return

k_client_service = gen_session_key()

ticket_service = encrypt(
    key_service,
    {
        "username": tgt_data.get("username"),
        "k_client_service": k_client_service.hex(),
        "valid_until": tgt_data.get("valid_until"),
    },
)

response = {
    "ticket_service": ticket_service,

```

```

        "k_client_service_enc": encrypt(
            k_client_tgs, {"k_client_service": k_client_service.hex()}
        ),
    }

    conn.sendall(json.dumps(response).encode())
    print(f"[TGS] Resposta enviada para o cliente {tgt_data.get('username')}")

```

4. Explicação detalhada do mecanismo utilizado para derivação de chaves a partir da senha do usuário (KDF)

- A senha **nunca** é armazenada nem trafega até os servidores Kerberos. No cadastro, gera-se um salt aleatório de 16 bytes e deriva-se a chave com **PBKDF2-HMAC-SHA256** (310.000 iterações, saída de 32 bytes); no banco guarda-se apenas `(salt, chave_derivada)`.

```

def derive_key(password: str, salt: bytes, length: int = 32) -> bytes:
    kdf = PBKDF2HMAC(
        algorithm=hashes.SHA256(),
        length=length,
        salt=salt,
        iterations=310_000,
    )
    return kdf.derive(password.encode())

```

5. Explicação detalhada do processo de obtenção e utilização dos tickets de autenticação

- Primeiro o usuário faz a requisição ao AS para conseguir o TGT e a `K_client_tgs`.
- Depois requisita o TGS com o TGT, o serviço (o chat) e um autenticador que garante que foi ele quem fez a requisição.

- O TGS responde com o ticket de serviço e a `K_client_service`. O backend, que é o próprio serviço, abre o ticket com a `K_chat`, confere a validade e a partir daí cria a sessão do usuário no chat.

```
def login(username: str, password: str) -> dict:
    cfg = current_app.config

    try:
        tgt, k_client_tgs = get_tgt(
            cfg["AS_HOST"], cfg["AS_PORT"], username, password)
        ticket_service, _k_client_service = get_service_ticket(
            cfg["TGS_HOST"], cfg["TGS_PORT"], tgt, k_client_tgs, SERVICE_NAME, username=username)
    except KerberosUnavailable as exc:
        logger.error(
            "login for %r failed: AS/TGS unreachable: %s",
            username, exc)
        raise LoginUnavailable("authentication service unavailable") from exc
    except KerberosError as exc:
        logger.info(
            "login failed for %r during AS/TGS exchange: %s",
            username, exc)
        raise LoginError("invalid credentials") from exc

    try:
        ticket_data = decrypt(cfg["CHAT_SERVICE_KEY"], ticket_service)
    except Exception as exc:
        logger.warning(
            "login failed for %r: could not verify service ticket: %s",
            username, exc)
```

```

        raise LoginError("invalid credentials") from exc

    if not check_expiration(ticket_data.get("valid_until")):
        logger.info("login failed for %r: expired service ticket", username)
        raise LoginError("invalid credentials")

    return {
        "username": ticket_data["username"],
        "valid_until": ticket_data["valid_until"],
    }

```

6. Explicação detalhada do mecanismo de autenticação mútua implementado

- A autenticação mútua no Kerberos decorre diretamente do uso de AES-GCM (cifra autenticada): só produz um texto cifrado que a outra parte decifra com sucesso quem conhece a chave correta.
- **Usuário ↔ AS.** O AS não recebe a senha; ele devolve `K_client_tgs` cifrada com a chave do usuário (derivada da senha).
 - **O cliente autentica o AS:** só o AS legítimo, que tem a chave do usuário no banco, conseguiria montar uma resposta que o cliente decifra com sucesso.
 - **O usuário passa a provar a senha:** só quem conhece a senha recupera `K_client_tgs`. Essa posse é *efetivamente provada* na etapa seguinte, ao montar o autenticador para o TGS.
- **Usuário ↔ TGS.** O cliente envia o TGT (opaco, cifrado com `K_AS_TGS`) e um autenticador `{username, timestamp}` cifrado com `K_client_tgs`.
 - **O TGS autentica o usuário:** ele decifra o TGT (provando que é o TGS legítimo, pois só ele e o AS têm `K_AS_TGS`), extrai `K_client_tgs` e decifra o autenticador. Se funciona e o `username` do autenticador bate com o do TGT, o cliente comprovadamente conhece `K_client_tgs` — logo, conhece a senha.

- **O cliente autentica o TGS:** a resposta traz `K_client_service` cifrada com `K_client_tgs`. Só um TGS capaz de abrir o TGT poderia conhecer essa chave e produzir uma resposta legível.

7. Descrição dos algoritmos criptográficos utilizados e justificativa de sua escolha

- **PBKDF2-HMAC-SHA256 (derivação de chave a partir da senha).**
 - 310.000 iterações, salt de 16 bytes, saída de 32 bytes.
 - O salt (único por usuário) evita rainbow tables; as muitas iterações deixam ataques de força bruta/dicionário lentos e caros.
- **AES-256-GCM (usado em todos os tickets, chaves de sessão e autenticadores).**
 - GCM é cifra autenticada: dá confidencialidade e integridade de uma vez só. Se a chave estiver errada ou o dado for adulterado, a decifração falha — é isso que sustenta a autenticação mútua (conseguir decifrar = provar que tem a chave).
 - Nonce aleatório de 12 bytes por operação.
- **os.urandom / secrets (aleatoriedade).**
 - Chaves de sessão, nonces, salts e tokens de sessão usam gerador criptograficamente seguro (CSPRNG).

8. Relato das principais dificuldades encontradas e dos aprendizados obtidos

- A maior dificuldade foi o próprio protocolo do Kerberos, que tem muitos passos e várias peculiaridades.
- Outra parte difícil, porque eu ainda não tinha o entendimento completo, foi como garantir a autenticação depois que o usuário termina todo o procedimento no Kerberos — isso gerou bastante incerteza até a ideia fechar.